

19. Parcourir : le total, le plus grand, le plus petit

Une boucle qui passe sur chaque case, et une variable qui retient ce qu'on a vu.

1. Ce qu'il faut comprendre

Presque tout ce qu'on fait avec un tableau suit le même modèle : **une variable de résultat, préparée avant la boucle**, puis **une boucle qui la met à jour à chaque case**.

Le total : on part de 0, et on ajoute chaque case. Attention au chapitre 1 : si le total dépasse 255, il déborde. Ici $12 + 40 + 7 + 99 + 25 = 183$, ça tient.

Le plus grand : on part de la **première case** (et pas de 0 !), puis on la remplace chaque fois qu'on trouve plus grand. On retient aussi **où** on l'a trouvé : `ouMax` — dans un jeu, c'est « quel joueur a gagné ».

Le plus petit : même chose avec `<`. Si l'on partait de 0 comme pour le total, aucune case ne serait jamais plus petite, et le résultat serait faux : **la valeur de départ est une vraie décision**.

Une seule boucle suffit pour les trois : on peut mettre à jour plusieurs résultats au même passage.

À toi : calcule la **moyenne** (le total divisé par le nombre de cases — division entière).

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **942 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      66 o ← la console, à l'allumage – la mise en route de la console
main()         206 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← ecran() – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() – écrit un nombre en chiffres
CopierTuiles   103 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         400 o ← CopierTuiles – 25 lettres sur 43
"TOTAL:"       6 o ← texte() – un texte gravé
"PLUS GRAND:"  11 o ← texte() – un texte gravé
"PAR LE JOUEUR" 13 o ← texte() – un texte gravé
"PLUS PETIT:"  11 o ← texte() – un texte gravé
scores_depart_0 5 o ← démarrage – une table gravée
```

Lettres gravées : ADEGIJLNOPRSTU0123456789:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 4    44 o sur place, dans main() + EcrireTexte (10 o)
nombre() × 4   40 o sur place, dans main() + EcrireNombre (33 o)
```

```
ecran()  x 2    21 o sur place, dans main() + AttendreVBlank (16 o)
image()   x 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : parcourt un tableau de 5 scores pour trouver
// le total, le plus grand (et qui l'a fait), et le plus petit.
//
// La méthode : une seule boucle passe sur chaque case, et met à jour
// trois « résultats en cours ».
```

```
#include <ecran>    // éteint ou rallume l'écran
#include <texte>     // écrit un texte à l'écran
#include <nombre>    // écrit un nombre en chiffres
```

2. UN TABLEAU — SCORES

```
uint8_t scores[5] = { 12, 40, 7, 99, 25 }; // scores[0] = 12 ... scores[4] = 25.
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    ecran(0);                               // Dessin écran éteint.
    uint8_t total = 0;                      // La somme, qui commence à 0.
    uint8_t plusGrand = scores[0];          // Le plus grand VU JUSQU'ICI : au départ, le premier.
    uint8_t ouMax = 0;                      // L'indice (le numéro) du plus grand.
    uint8_t plusPetit = scores[0];          // Le plus petit vu jusqu'ici : au départ, le premier.

    for (uint8_t i = 0; i < 5; i++) { // i = 0 à 4 : chaque case, une fois.
        total = total + scores[i];         // On ajoute la case au total.
                                           // 0 + 12 = 12, + 40 = 52, + 7 = 59, + 99 = 158, + 25 = 183.

        if (scores[i] > plusGrand) {        // Cette case bat-elle le record ?
            plusGrand = scores[i];          // oui : c'est le nouveau plus grand,
            ouMax = i;                      // et on retient où il est.
        }                                  // (40 bat 12 ; puis 99 bat 40 : ouMax = 3.)
        if (scores[i] < plusPetit) {        // Plus petit que le plus petit vu ?
            plusPetit = scores[i];          // oui : on le garde. (7 bat 12.)
        }
    }

    texte(1, 3, "TOTAL:");                 nombre(14, 3, total);      // 183
    texte(1, 5, "PLUS GRAND:");             nombre(14, 5, plusGrand);    // 099
    texte(1, 6, "PAR LE JOUEUR");           nombre(16, 6, ouMax, 1);     // 3
    texte(1, 8, "PLUS PETIT:");             nombre(14, 8, plusPetit);    // 007

    ecran(1);                               // On rallume.

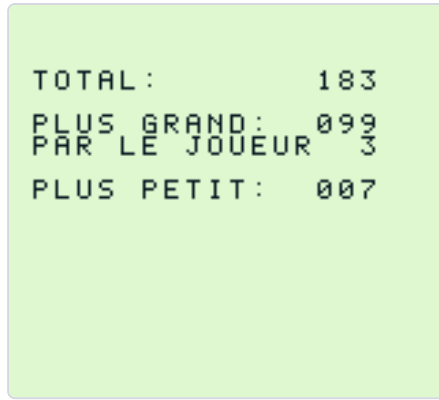
    while (true) {                          // La boucle du jeu.
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Total 183, plus grand 099 (joueur 3), plus petit 007.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le total est 183
- ✓ le plus grand est 99
- ✓ trouvé chez le joueur 3
- ✓ le plus petit est 7

← [18. Un tableau : plusieurs cases sous un seul nom](#)

[sommaire](#)

[20. Une table gravée : dessiner un niveau](#) →